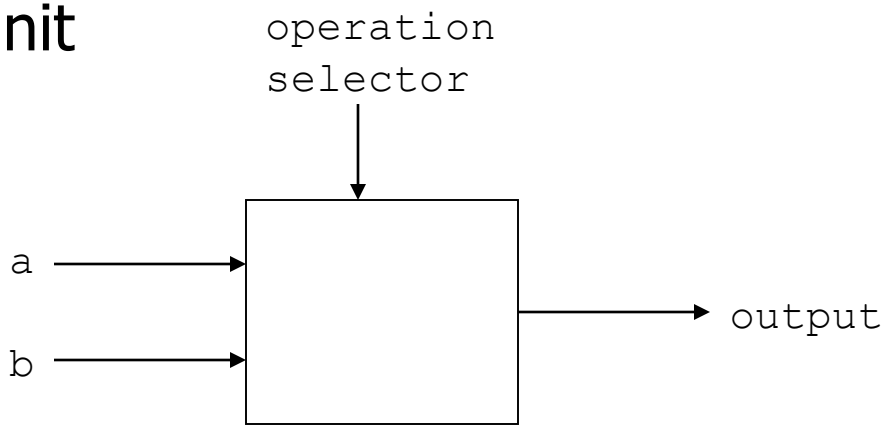
A Simple Multi-Function Logic Unit

build a logic unit to support the `and` and `or` function.

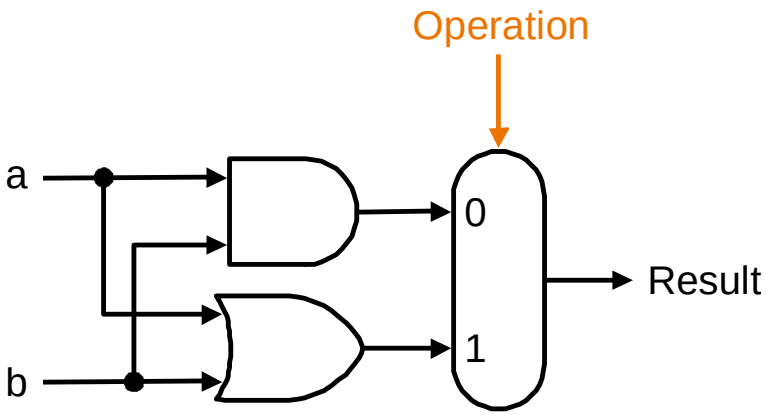
1-bit unit



Possible implementation using a *multiplexor* :

Implementation with a Multiplexor

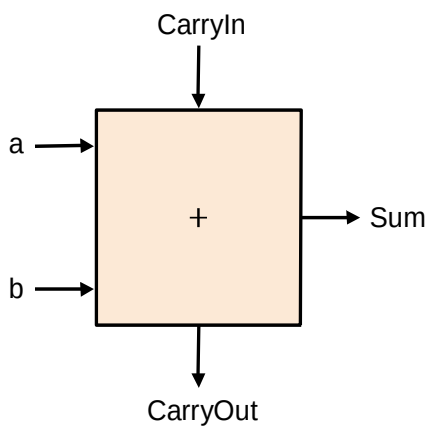
Selects one of the inputs to be the output based on a control input



build ALU using a MUX (multiplexor):

Implementation

1-bit ALU for addition:



$$c_{out} = a.b + a.c_{in} + b.c_{in}$$

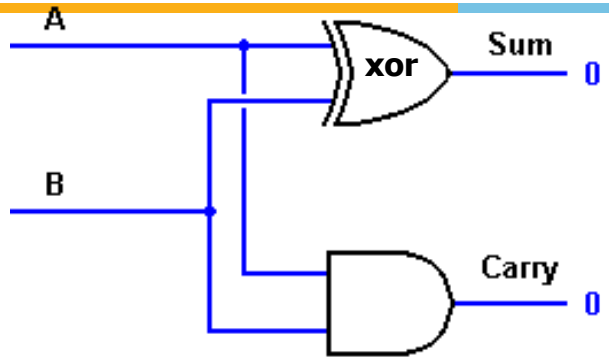
$$\begin{aligned} sum &= a.\overline{b}.\overline{c_{in}} + \overline{a}.b.\overline{c_{in}} + \\ &\quad \overline{a}.\overline{b}.c_{in} + a.b.c_{in} \\ &= a \oplus b \oplus c_{in} \end{aligned}$$

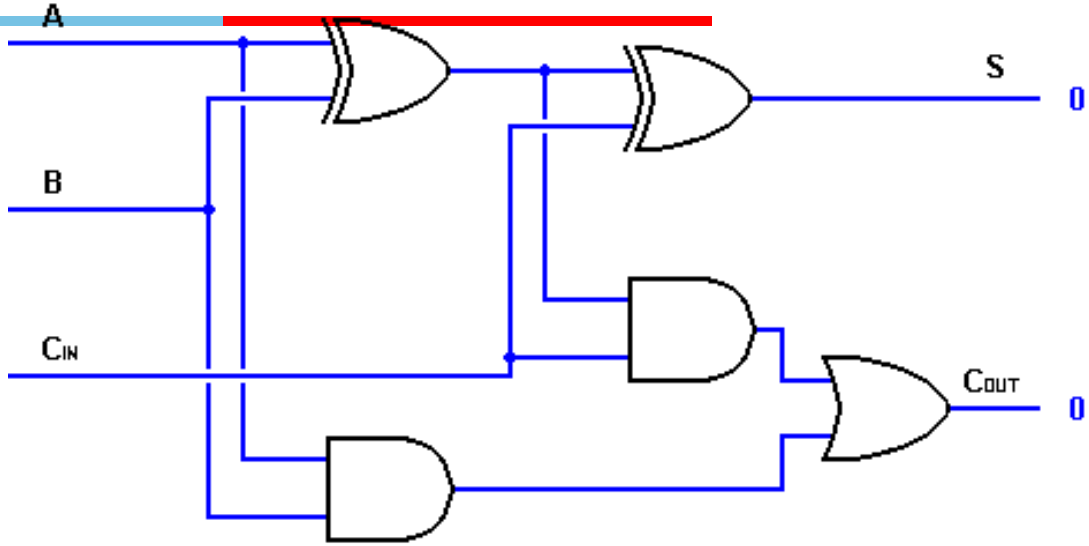
How could we build a 1-bit ALU for add, AND, OR

How could we build a 32-bit ALU?

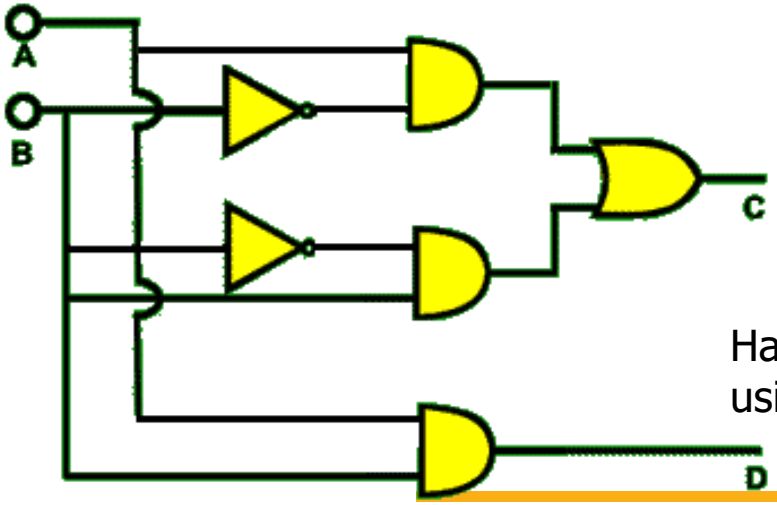
Some of the Design questions.
do not want too many inputs to a single gate
do not want to have to go through too many gates (= levels)

1-bit Adder Logic



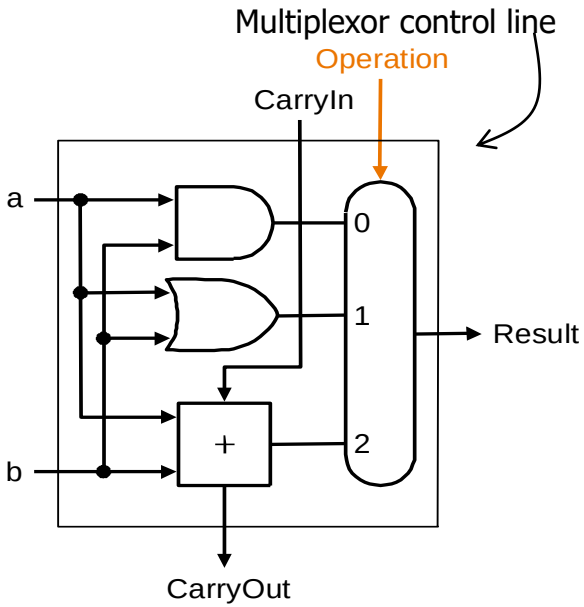


Full-adder from 2 half-adders and an or gate



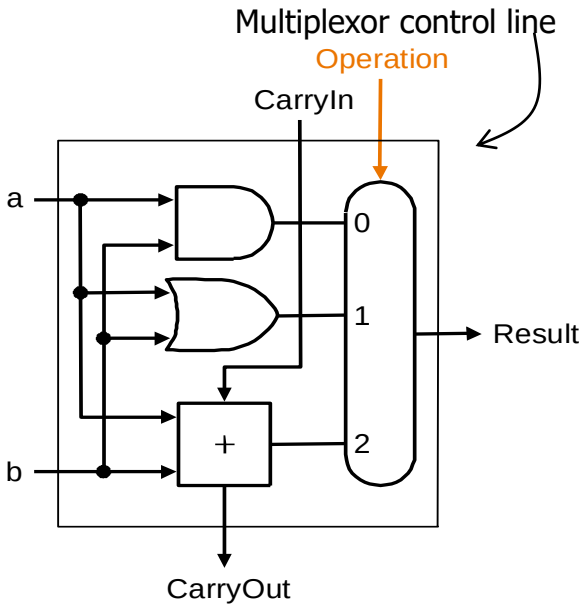
Half-adder with the xor gate replaced by primitive gates using the equation $A \oplus B = A.\overline{B} + \overline{A}.B$

Building a 32-bit ALU

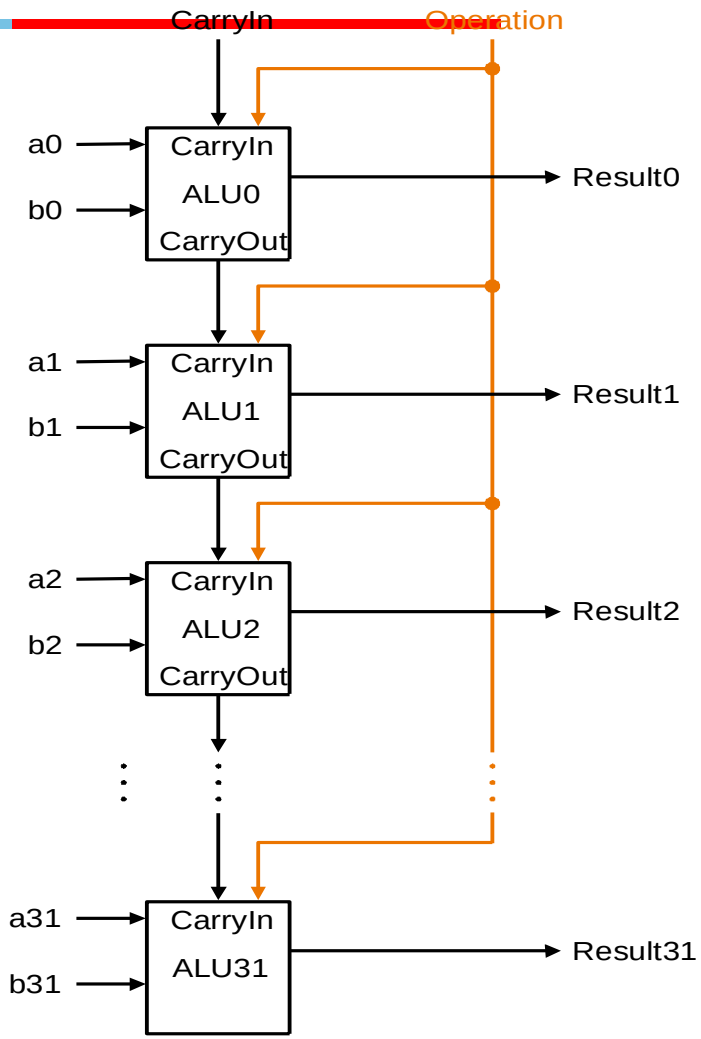


1-bit ALU for AND, OR and add

Building a 32-bit ALU



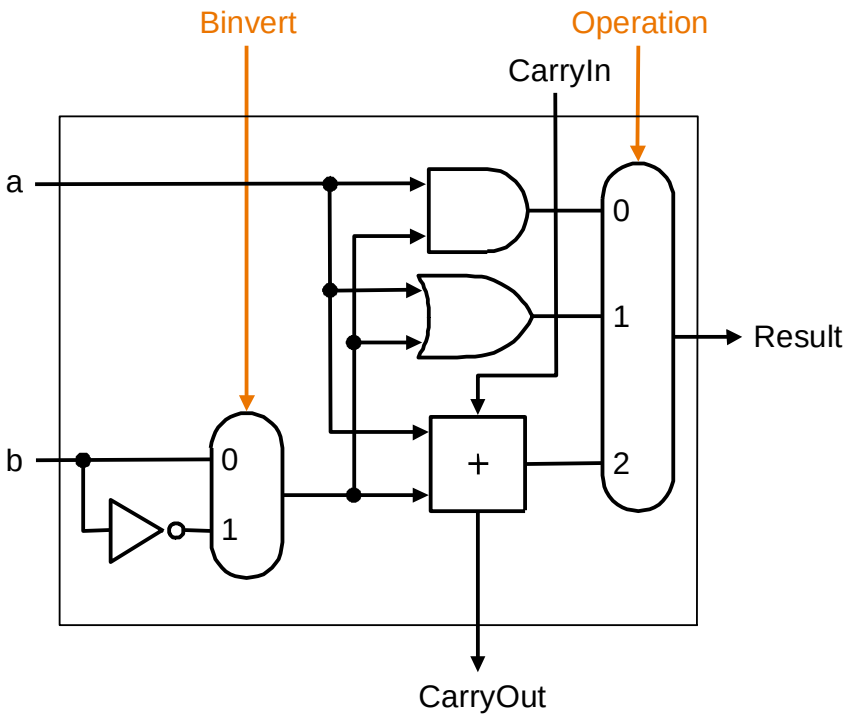
1-bit ALU for AND, OR and add



Ripple-Carry Logic for 32-bit ALU

Subtraction (a – b)

Two's complement approach: just negate b and add.
invert each bit of b and set Carry In to *least significant bit* (ALU0) to 1



Tailoring the ALU to MIPS: Test for Less-than and Equality

Tailoring the ALU to MIPS: Test for Less-than and Equality

Need to support the *set-on-less-than* instruction
e.g., `slt $t0, $t3, $t4`
`slt` is an *R-type instruction* that produces 1 if rs < rt and 0 otherwise
use subtraction: rs < rt $\Leftrightarrow$ rs – rt < 0. msb of negative number is 1

Tailoring the ALU to MIPS: Test for Less-than and Equality

two cases after subtraction rs – rt:

<u>if no overflow</u> then rs < rt $\Leftrightarrow$ most significant bit of rs – rt = 1

<u>if overflow</u> then rs < rt $\Leftrightarrow$ most significant bit of rs – rt = 0

e.g., $5_{ten} - 6_{ten}$ = 0101 – 0110 = 0101 + 1010 = 1111 (ok!)

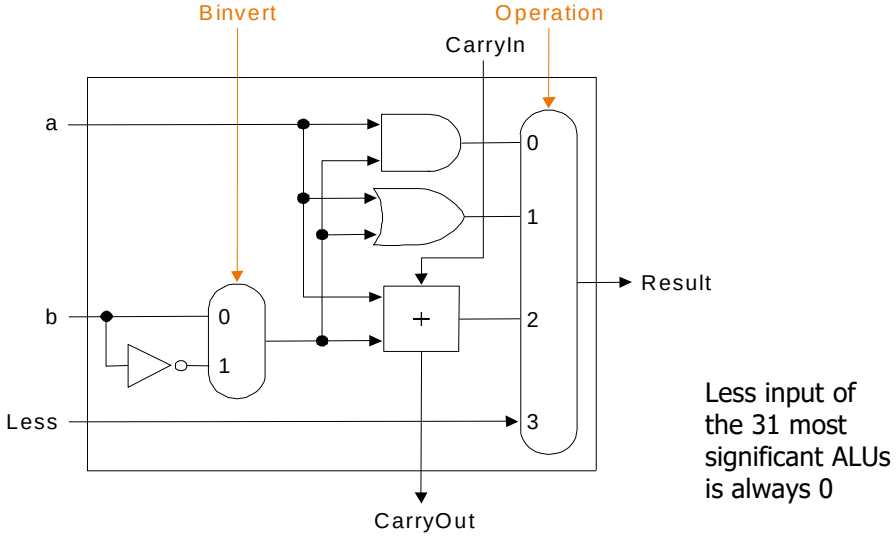
$-7_{ten} - 6_{ten}$ = 1001 – 0110 = 1001 + 1010 = 0011 (overflow!)

therefore

set bit = msb of rs – rt $\oplus$ overflow bit

where *set bit*, which is output from ALU31, gives the result of `slt`

set bit is sent from ALU31 to ALU0 as the *Less* bit at ALU0; all other Less bits are hardwired 0; so Less is the 32-bit result of `slt`

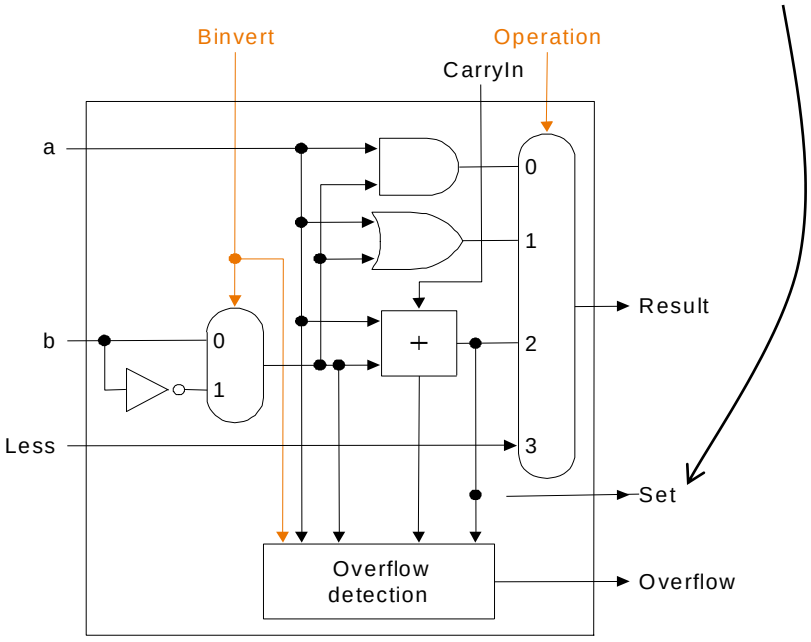


Less input of the 31 most significant ALUs is always 0

a.

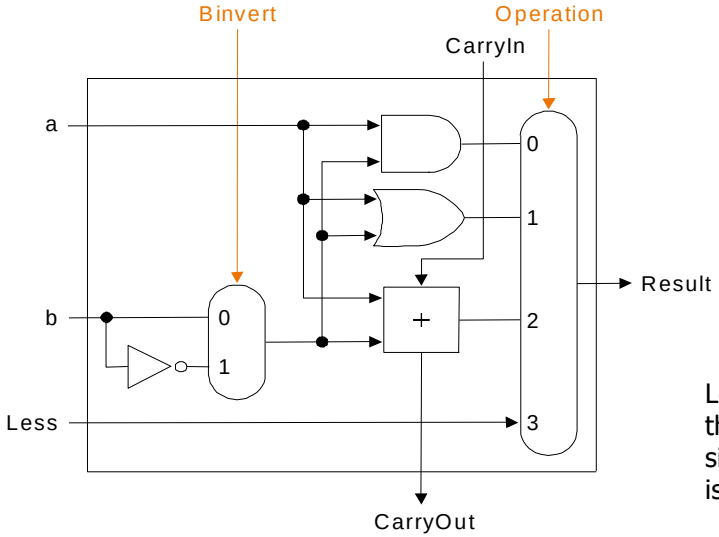
1- bit ALU for the 31 least significant bits

Extra set bit, to be routed to the Less input of the least significant 1-bit ALU, is computed from the most significant Result bit and the Overflow bit



b.

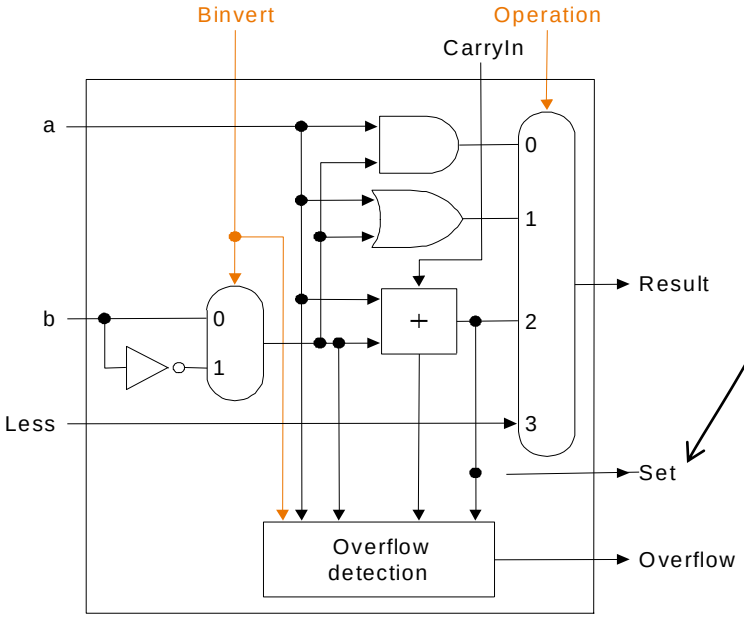
1-bit ALU for the most significant bit



a.

1- bit ALU for the 31 least significant bits

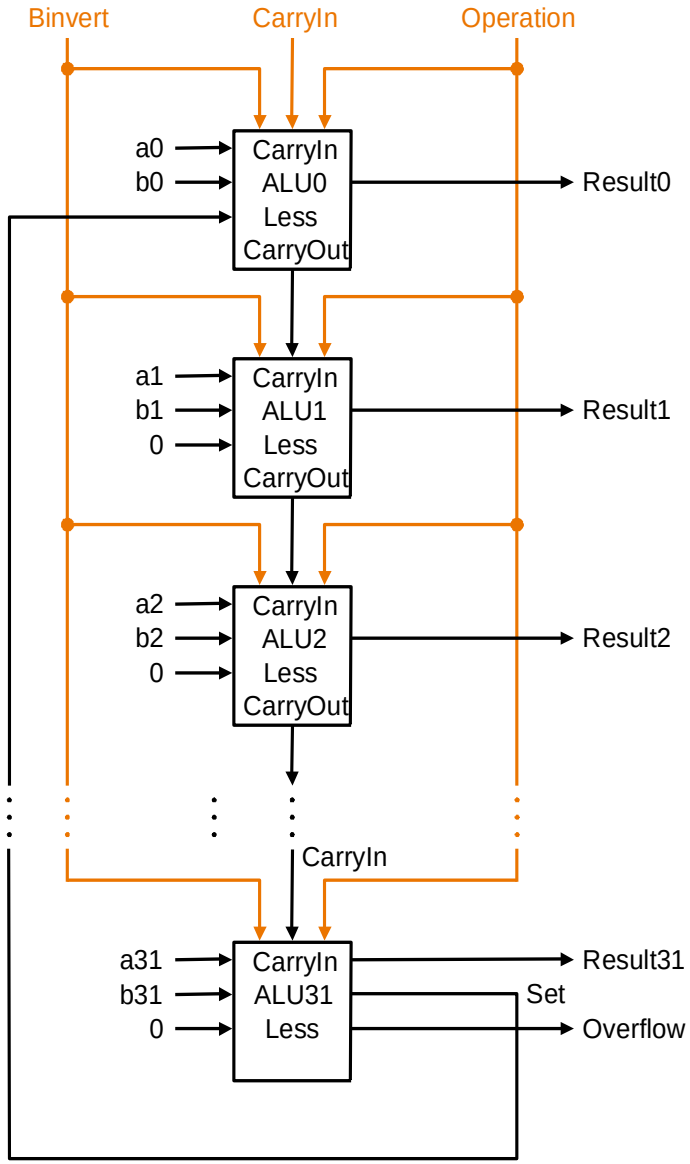
Extra set bit, to be routed to the Less input of the least significant 1-bit ALU, is computed from the most significant Result bit and the Overflow bit



b.

1-bit ALU for the most significant bit

Less input of the 31 most significant ALUs is always 0

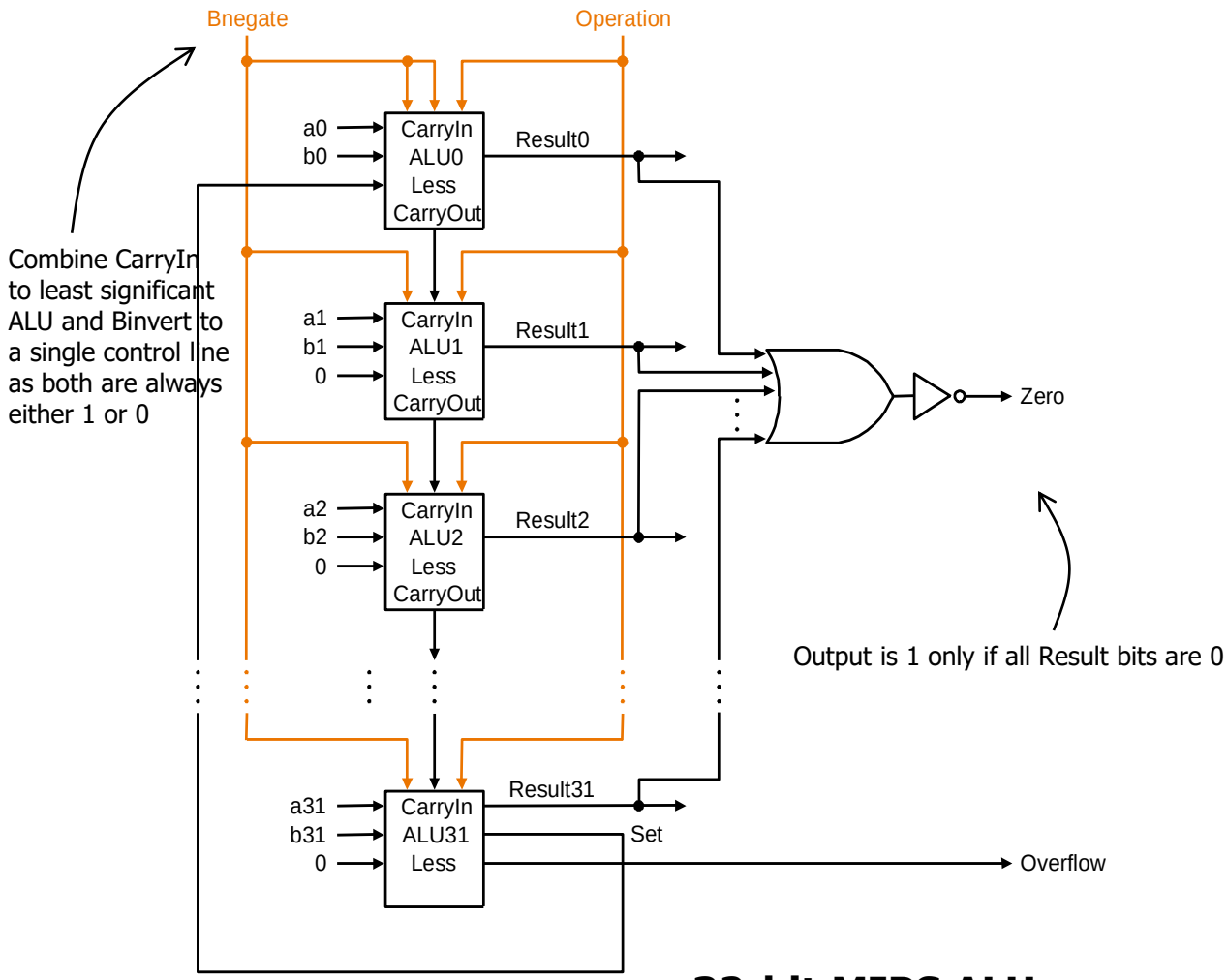


32-bit ALU from 31 copies of ALU at top left and 1 copy of ALU at bottom left in the most significant position

Tailoring the ALU to MIPS: Test for Less-than and Equality

- What about logic for the *overflow bit* ?
  - overflow bit = carry *in to* msb $\oplus$ carry *out of* msb
  - logic for overflow detection therefore can be put in to ALU31
- Need to support *test for equality*
  - e.g., `beq $t5, $t6, label`
  - use subtraction: rs - rt = 0 $\Leftrightarrow$ rs = rt

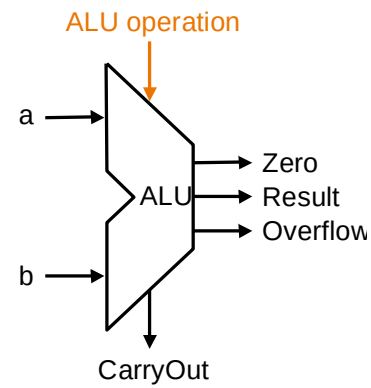
Supporting Test for Equality



32-bit MIPS ALU

ALU control lines

Bneg-ate	Oper-ation	Func-tion
0	00	and
0	01	or
0	10	add
1	10	sub
1	11	slt



Symbol representing ALU